

Agent skill evolution under ecological selection: deliberation accelerated variation, layered validation, and memory inheritance in cluster operations

Abstract This paper addresses the manual construction of agent skill variants, the lack of systematic selection for candidates, and the difficulty of carrying capability across agents, taking variation, selection, retention, and inheritance in ecological evolution as the underlying logic of agent-skill evolution. Deliberation accelerates useful variation. The Group Deliberation Space (Plaza) uses an ORID agenda, a 12-Niche role-function topology, and ritual signals to increase meaningful knowledge variants, while DART-Net and the Temporal Skill Extractor (TSE) convert deliberation transcripts into fixed-field candidate skills with provenance links. Six gates covering schema, provenance, tools, execution, safety, and regression, together with incumbent-challenger competition, perform engineering pre-selection under safety and non-degradation constraints. An ecological-pressure environment operationalizes environmental selection through resource abundance, predation pressure, and niche capacity, returning differential-persistence evidence to the next round of variation. A four-layer memory model and the Seal-Will-Export-Import (SWEI) protocol provide post-selection retention and inheritance. In cluster-operations experiments, the complete Plaza configuration improves candidate quality; local DART-Net/TSE extraction averages 4.81ms; SkillRouter achieves a 100% Top-5 hit rate with 1.94 ms average latency; and four closed-loop cycles increase the agent skill count from 5 to 9 while the internal quality score Q rises from 0.42 to 0.78. The ecological-pressure results show a sample-level inverted-U relationship between collaboration frequency and resource abundance. These findings show the system operationalizes environmental-selection signals. However, limited replication do not establish a general law of natural selection. Ecological selection is the basis of agent skill evolution, deliberation is a variation accelerator, and memory inheritance carries selected capability forward.

Keywords agent skill evolution; ecological selection; deliberation-accelerated variation; layered validation; memory inheritance; multi-agent systems; cluster operations

1. Introduction

Current agents can invoke tools, execute workflows, and complete tasks, yet their skills still depend largely on external design and maintenance. The philosophical starting point of this work is that ecological regularity precedes engineering mechanism. In natural systems, capability persists through variation, environmental selection, differential retention, and inheritance. For agents, candidate skills must likewise be evaluated under task, safety, and environmental pressure and carried to successors before ordinary version updating can be called evolution. Deliberation does not create this regularity; it increases the rate of meaningful variation and reduces blind trial and error. Three connected gaps must therefore be addressed.

Capability gap 1: agents can execute tasks but cannot systematically form reusable skills. Existing multi-agent frameworks have demonstrated strong potential in multi-turn interaction, role specialization, and tool use [1-6]. However, skills are typically pre-authored by developers or domain experts and statically injected as prompts, function definitions, workflows, or rule templates. As the skill inventory grows, manual authoring, review, and maintenance become a major scalability bottleneck.

Capability gap 2: agents can generate candidates but cannot determine whether they are trustworthy. Group deliberation can contain misunderstandings, omissions, or erroneous consensus, while extraction models can misalign evidence or produce inconsistent field combinations. Candidate skills therefore require systematic checks of factual grounding, executability, and release conditions rather than being accepted directly after generation.

Capability gap 3: agents can accumulate experience but cannot reliably carry it across upgrades. Skill discovery and evolution depend on historical context, whereas a single large-language-model invocation is essentially stateless. Concatenating prior messages or producing summaries does not reliably distinguish completed events, unfinished intentions, and risk states. When an agent is upgraded or replaced, the successor may receive the same tools and skills without knowing how those capabilities were formed, under which conditions they are valid, or why they were revised.

The three capability gaps form one evolutionary chain. Without variation generation, the system has no skill differences on which selection can act. Without validation, execution, and environmental pressure, it cannot produce differential selection. Without version retention and memory transfer, selected outcomes cannot survive agent replacement. Deliberation and extraction sit at the variation end: they accelerate and structure candidate production rather than serving as the underlying evolutionary driver. The challenge is not to relabel ordinary updating as evolution, but to provide executable and auditable mechanisms for variation, selection, retention, and inheritance.

We take variation, selection, retention, and inheritance in natural selection as the underlying evolutionary regularity and engineer it into four stages.

(1) Deliberation-accelerated variation. Agents with different knowledge functions supplement facts, challenge risks, compare alternatives, and reach decisions in Plaza. DART-Net/TSE then extracts candidate skill variants with names, tools, steps, constraints, and provenance evidence.

(2) Constrained pre-selection. Candidate variants pass through six-layer validation, controlled execution, and incumbent-challenger competition. Safety, key metrics, and non-degradation evidence first remove infeasible, unsafe, or clearly degrading variants.

(3) Ecological environmental selection. Published skills and their combinations enter a simplified environment with resource, predation, and niche constraints. Differential persistence is returned to the next round of variation and version competition.

(4) Capability retention and inheritance. Published versions, execution evidence, events, perceptions, intentions, and risk states are sealed, filtered, verified, and transferred to successors so that environmentally selected capability enters the next evolutionary round.

Within this framework, agents can actively discover skills, evaluate and revise them using evidence, and preserve relevant experience across upgrades or replacement. The agent therefore participates in the full skill lifecycle, spanning knowledge formation, candidate generation, trusted validation, task execution, version evolution, and cross-agent inheritance.

The main contributions are as follows.

- (1) We establish an agent-skill evolution model grounded in ecological selection and map variation, constrained pre-selection, environmental selection, differential retention, and memory inheritance into one executable and traceable engineering chain.
- (2) We propose Plaza, where the ORID agenda, 12-Niche role-function positions, and ritual signals organize factual supplementation, risk challenges, alternative comparison, and decision closure, positioning deliberation as an accelerator of useful variation.
- (3) We propose DART-Net and TSE to convert deliberation transcripts into fixed-schema, provenance-linked candidate skills through temporal modeling, field queries, and candidate-source indexing, making accelerated variants comparable in subsequent selection.
- (4) Section 5 implements two levels of selection. Six-layer validation, controlled execution, and version competition provide engineering pre-selection under safety and non-degradation constraints; ecological pressure then observes differential persistence of published skills and combinations under resource change and multigenerational runs and returns the evidence to variation.
- (5) We establish a four-layer memory architecture for events, perceptions, intentions, and risk tendencies, and introduce SWEI as the inheritance mechanism that retains selected capability, validity conditions, failures, and risk records.
- (6) We organize cluster-operations experiments around deliberation-accelerated variation, constrained pre-selection, ecological environmental selection, and capability inheritance, covering candidate formation, millisecond-scale

extraction, governance, differential-persistence records, controlled transfer, and robustness while making the exploratory evidence boundary explicit.

Ecological evolution is not an added experiment outside the loop; it is the underlying regularity that explains differential retention of skills. Deliberation accelerates variation, the six gates perform human-constrained pre-selection, ecological pressure represents environmental selection, and memory inheritance carries the result forward. The current experiment shows only that this regularity has been operationalized and produces recordable persistence signals. It does not equate a short simulation with empirical proof of a universal law of natural selection. Formal skill states remain governed by the six gates and version competition.

2. Related work

2.1. From multi-agent deliberation to skill knowledge production

Parliamentary and meeting procedures emphasize that agenda setting, speaking rights, objection handling, and voting rules shape group discussion [7]. Participatory decision-making further argues that diverse perspectives, staged discussion, and neutral facilitation help groups move from information exchange to disagreement exploration and executable decisions [8-9]. In the large-language-model literature, multi-agent debate, role-based discussion, and round-table consensus have been used to improve divergent thinking, cross-checking, and reasoning quality [1-3]. These studies indicate that deliberation quality depends not only on what participants say but also on agenda structure, role allocation, and the way disagreement is handled.

Most existing multi-agent deliberation systems, however, are optimized for solving the current task or improving a one-shot answer. After the discussion ends, facts, objections, tool suggestions, failure boundaries, and action steps usually remain embedded in free-form dialogue, without stable fields, provenance identifiers, or lifecycle states that would allow them to become reusable skills.

Our focus is therefore not deliberation for its own sake, but how agents can use deliberation to discover reusable knowledge, transform it into verifiable skills, and then continue skill evolution.

2.2. Information extraction, skill libraries, and evidence-driven evolution

Extracting skills from deliberation transcripts requires modeling temporal relations among utterances, locating field-level evidence, and generating objects that satisfy structural constraints. Temporal convolutional networks provide a parallel convolutional approach to sequence modeling [10]. Longformer illustrates the need to control context cost for long documents [11], while GoLLIE and BART provide foundations for schema-oriented information extraction and constrained generation [12-13]. These methods support temporal modeling, field localization, and structured generation, but information extraction alone does not define a complete skill lifecycle: it does not determine whether a candidate is trustworthy, when it should be published, how execution evidence should revise it, or how provenance continuity should be maintained after revision.

In the agent-skill literature, Voyager shows that an agent can continually extend a skill library through environmental interaction [14], demonstrating that skills need not be fully hand-authored in advance. We focus further on how candidate skills emerge from multi-role deliberation with provenance evidence, how their trustworthiness is assessed, and how usage evidence drives constrained non-degrading evolution.

2.3. Agent memory and cross-agent inheritance

Generative Agents and the CoALA architecture highlight the importance of modular memory, retrieval, and reflection for persistent agent behavior [6,15]. Event records, semantic knowledge, and reflections can support behavioral continuity over multiple tasks or long horizons. Most memory work, however, concentrates on storage, retrieval, and context construction inside a single agent rather than treating memory state, candidate skills, formal versions, execution records, and successor transfer as parts of one controlled lifecycle.

We therefore place the experience accumulated during skill formation, validation, and use inside the memory-management process, so that relevant experience can survive upgrades, replacements, and cross-team transfers while retaining provenance.

2.4. Ecological simulation and evolutionary analysis

Natural selection requires more than change: variants must be retained differentially under environmental constraints and passed into later states. Work on dual inheritance, digital evolution, resource competition, and long-running computational populations provides methodological support for this structure [16-21]. We therefore treat ecological evolution as the theoretical starting point rather than an add-on. Deliberation only accelerates skill variation, layered validation performs human-constrained pre-selection, ecological pressure represents environmental selection, and memory inheritance retains selected outcomes and provenance. The short ecological simulation tests whether this mapping can operate and produce recordable signals; it is not treated as proof of a universal biological law.

3. Skill-evolution loop and data model

3.1. Closed-loop workflow

The workflow follows the underlying logic of ecological selection rather than combining coequal modules. Deliberation and extraction accelerate variation, layered validation and version competition perform human-constrained pre-selection, ecological pressure provides environmental selection, and the version repository and memory system provide differential retention and inheritance.

As shown in Fig. 1, the system cycles through deliberation, extraction, validation and release, assignment and execution, usage feedback, skill updating, memory updating, and renewed deliberation. Multi-role agents produce a Plaza transcript, DART-Net/TSE generates candidate skills and evidence spans, and the six gates plus version competition first remove infeasible, unsafe, or degrading variants. Pre-selected skills execute tasks and generate usage records. Ecological selection is implemented inside the figure's skill-update/fitness stage: environmental pressure produces differential persistence and returns the evidence to the next deliberation round. Deliberation is therefore the variation accelerator, whereas ecological selection explains why a skill or combination is retained or removed.

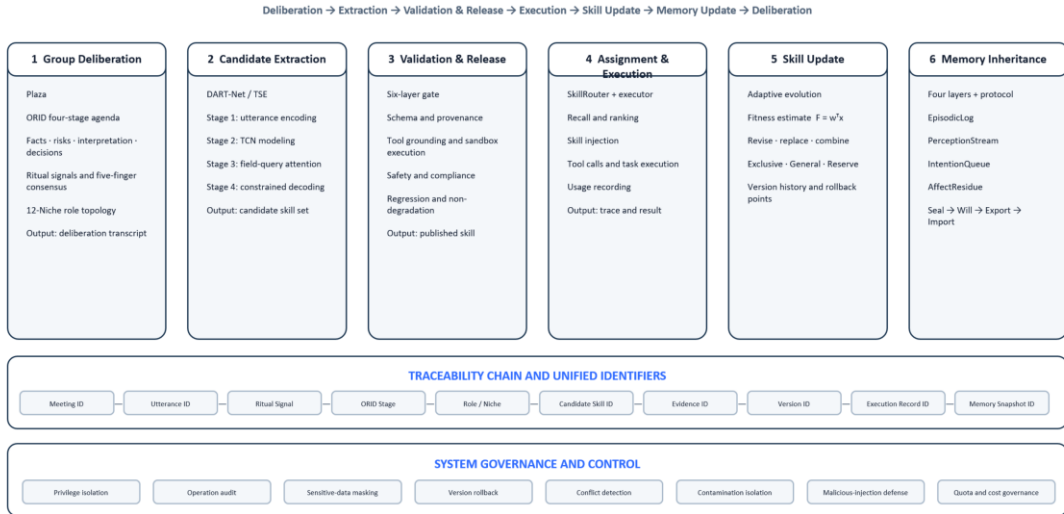


Figure 1: Skill evolution under ecological selection: deliberation accelerates variation, validation constrains pre-selection, and memory enables inheritance.

3.2. Joint state and provenance relations

To avoid connecting modules only through opaque interfaces, we define four state classes: deliberation, representation, skill, and memory. At closed-loop cycle k , the joint state is defined in Eq. (1).

$$Z_k = (p_k, d_k, s_k, m_k) \quad (1)$$

Here, p_k is the deliberation state, containing utterances, agenda stages, role-to-Niche mappings, ritual signals, challenge routes, and consensus outcomes; d_k is the representation state, containing utterance encodings, temporal hidden representations, field representations, and candidate provenance indices; s_k is the skill state, containing candidate structure, validation results, versions, and lifecycle state; and m_k is the memory state, containing events, perceptions, intentions, and risk tendencies.

Task, agent, timestamp, and provenance identifiers connect the four states into queryable snapshots. Section 4 describes the conversion from deliberation state to candidate skill variants; Section 5 describes layered validation, version competition, and skill evolution under ecological pressure; Section 6 describes memory retention and inheritance of selected outcomes.

The system also allocates identifiers to deliberations, candidate skills, published skills, execution records, sealed snapshots, and transfer packages. Candidate skills reference their source deliberation and evidence spans; published skills retain version relations and publication/update rationale; execution records refer to the invoked skill version, tool-call trace, and outcome; sealed snapshots preserve pre-transfer memory state; and transfer packages record source agent, beneficiary agent, inheritance scope, and conflict policy. These relations support validation, rollback, and memory transfer.

4. Deliberation-accelerated skill variation and candidate extraction

This section explains how a human-designed deliberation mechanism accelerates the formation of skill-knowledge variants and converts transcripts into comparable candidate skills. Deliberation expands meaningful variation and preserves provenance, but it does not determine persistence. The variants then enter layered pre-selection and ecological evolution in Section 5.

Let the k -th closed-loop cycle contain n_k temporally ordered utterances. The deliberation state is defined in Eq. (2).

$$p_k = (U_k, \omega_k, \eta_k, \sigma_k, \Gamma_k, r_k, \chi_k) \quad (2)$$

In Eq. (2), U_k is the ordered utterance sequence; ω_k is the ORID-stage sequence; η_k is the dynamic mapping among agents, roles, and Niche positions; σ_k is the ritual-signal sequence; Γ_k records agenda progression and challenge routing; r_k is the vector of five-finger consensus scores; and χ_k is the final consensus and decision state. Each utterance additionally records its text, agent identifier, role, Niche identifier, agenda stage, ritual signal, and time/order information.

Thus, the deliberation state is not an unstructured conversation: it explicitly combines content, agenda stage, knowledge-function position, deliberative action, challenge routing, and consensus results.

4.1. Four-stage ORID agenda

Plaza adopts Stanfield's ORID focused-conversation method [22] and organizes deliberation as Objective $\rightarrow$ Reflective $\rightarrow$ Interpretive $\rightarrow$ Decisional. The Objective stage is the factual layer, where a factual facilitator elicits verifiable facts, environment states, and available tools. The Reflective stage is operationalized as a risk-reflection layer that identifies uncertainty, potential cost, failure modes, abnormal conditions, and rollback boundaries. The Interpretive stage is a solution-debate layer in which an analysis facilitator compares candidate procedures, causal chains, and alternatives. The Decisional stage is led by a decision facilitator who consolidates candidate solutions and conducts five-finger consensus voting. The agenda and its mapping to skill information are illustrated in Fig. 3.

Let A_{ORID} denote the ORID stage set and let $\omega_{k,t}$ be the stage label of utterance $u_{k,t}$. Stage membership, forward progression, and backward routing are represented by Eq. (3).

$$\begin{aligned} u_{k,t} \in a_U^k &\Leftrightarrow \omega_{k,t} = a, \quad a \in A_{ORID} \\ \omega_{k,t+1} &= \Phi(\omega_{k,t}, \sigma_{k,t}; \Gamma_k) \end{aligned} \quad (3)$$

In Eq. (3), a_{ij}^k is the set of utterances assigned to stage a and Φ is the agenda-state transition function. In the absence of blocking objections, the agenda normally advances $O \rightarrow R \rightarrow I \rightarrow D$. When an utterance carries a CHALLENGE signal, Γ_k returns the factual, risk-reflection, or solution-debate stage that must be supplemented or revised.

The complete ORID agenda places facts, risks, interpretations, and actions into relatively stable semantic regions, allowing tool conditions, preconditions, failure boundaries, execution steps, and decision thresholds to be recorded separately and tagged with their agenda stages. The agenda organizes deliberative information and reflects participant acceptance; it does not replace provenance checks, tool validation, safety review, or execution-based evaluation.

4.2. 12-Niche role-function coverage and challenge routing

The 12-Niche role topology borrows the architectural niche metaphor [23] to map knowledge functions to relatively independent positions. “12” is used as a topology name rather than a requirement that exactly twelve agents participate. Inner-ring Niches primarily hold direct execution knowledge such as goals, tools, steps, and preconditions; middle-ring Niches focus on detection conditions, failure modes, risk constraints, and rollback boundaries; outer-ring Niches retain provenance, version, audit, and change information. Facilitators control agenda progression and do not necessarily occupy a Niche.

Let A_k , R_k , and N_k denote the participating agents, role set, and activated Niche positions in cycle k . The agent-role-Niche mapping and CHALLENGE routing are represented in Eq. (4).

$$\eta_k \subseteq A_k \times R_k \times N_k$$

$$(v_{k,t}, \omega_{k,t}, CHALLENGE, target_{\omega}^{k,t}) \in \Gamma_k \quad (4)$$

The mapping η_k is many-to-many: an agent may cover multiple Niches, and a Niche may be shared by several agents. $v_{k,t}$ denotes the Niche assigned to utterance $u_{k,t}$, and $target_{\omega}^{k,t}$ is the ORID stage to which a challenge is routed. A CHALLENGE originating from a middle- or outer-ring utterance is routed using both its Niche and current agenda stage, and is converted into factual supplementation, risk constraints, tool replacement, procedure revision, or rollback rules. Every candidate field is bound to the originating agent, role, Niche, agenda stage, and utterance identifier, preserving provenance.

Figure 2 shows the Plaza implementation. Agents are arranged around concentric knowledge-function rings; the center presents the current issue and role relations; and the discussion panel records rounds, role-specific utterances, and execution status. This interface places multi-role participation, Niche positioning, and agenda progression in one operating environment, providing system support for challenge routing, deliberation logging, and downstream skill extraction.

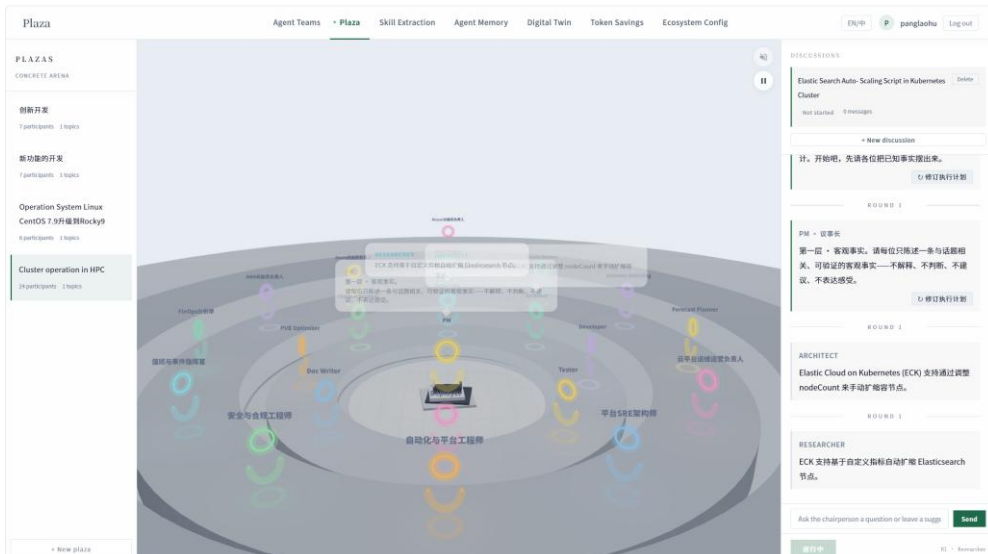


Figure 2: System implementation of the Plaza group-deliberation space and multi-role Niche layout.

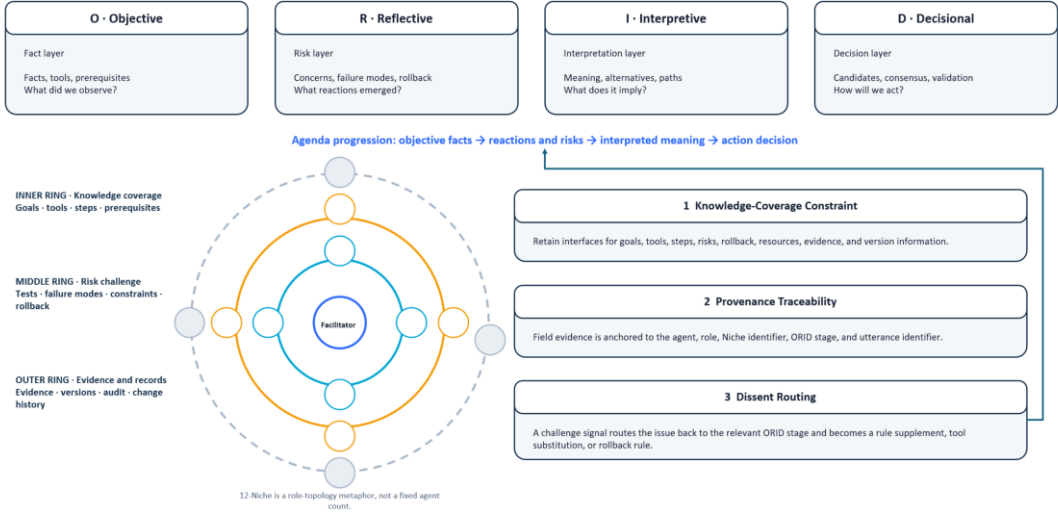


Figure 3: Four-stage ORID agenda and 12-Niche role topology.

4.3. Ritual signals and five-finger consensus

Each utterance carries a ritual signal from $\{\text{SUPPLEMENT, CHALLENGE, AGREE, COURT, DIGRESS}\}$. These signals respectively add facts or constraints, challenge assumptions or failure modes, support an existing proposal, move a visible disagreement into formal debate, or mark and terminate an off-topic branch. Their primary value is to retain the type of deliberative act, the originating role, and its agenda stage so that objections can be traced across agent, role, Niche, agenda stage, and utterance identifiers.

Suppose M_k participants submit five-finger consensus scores in cycle k . The score vector is $r_k = (r_{k,1}, \dots, r_{k,M_k})$, where $r_{k,j} \in \{1,2,3,4,5\}$. Let ρ_k be the proportion of scores no lower than 3 and μ_k the mean score. The gates for entering validation and declaring strong consensus are defined in Eq. (5).

$$\begin{aligned}
 \rho_k &= \frac{1}{M_k} \sum_{j=1}^{M_k} 1[r_{k,j} \geq 3] \\
 \mu_k &= \frac{1}{M_k} \sum_{j=1}^{M_k} r_{k,j} \\
 cons_g^k &= 1[\min_j r_{k,j} > 1 \wedge \rho_k \geq \theta_\rho] \\
 strong_g^k &= 1[cons_g^k = 1 \wedge \mu_k \geq \theta_\mu]
 \end{aligned} \tag{5}$$

Here, θ_ρ and θ_μ are the support-ratio and strong-consensus thresholds. $cons_g^k = 1$ indicates that the proposal is not blocked and has sufficient support to proceed to validation; $strong_g^k = 1$ indicates that the proposal additionally satisfies the strong-consensus criterion. The deliberation result χ_k records the corresponding consensus state.

4.4. DART-Net and TSE for candidate skill extraction

Sections 4.1–4.3 define the deliberation state. DART-Net and TSE receive p_k together with utterance order, ORID stage, role, Niche position, ritual signal, challenge routing, and consensus information. DART-Net transforms p_k into an intermediate representation d_k , and TSE combines d_k with source indices to generate the candidate skill state $cand_s^k$, as summarized in Eq. (6).

$$p_k \xrightarrow{DART-Net} d_k \xrightarrow{TSE} cand_s^k \quad (6)$$

The representation state d_k stores utterance encodings, temporal hidden representations, field representations, and candidate provenance indices; $cand_s^k$ denotes a candidate skill that has not yet passed the validation gates in Section 5.

DART-Net is a representation architecture for group-deliberation transcripts with four stages: utterance feature encoding, temporal-relation modeling, field-query and candidate-provenance aggregation, and schema-constrained decoding. TSE orchestrates these stages and converts the representations into candidate skills with provenance relations. The extraction pipeline retains temporal order, cross-stage dependencies, and field-to-utterance source indices, as illustrated in Fig. 4.

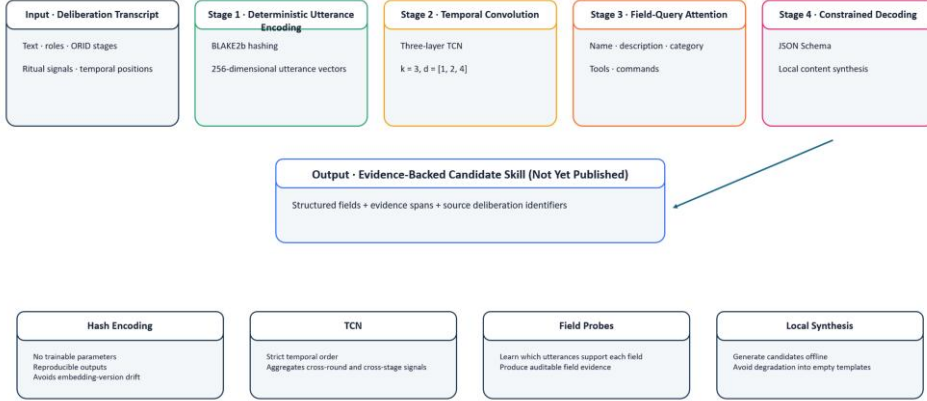


Figure 4: DART-Net architecture and the TSE candidate skill extraction pipeline.

4.4.1. Utterance feature encoding

For each utterance, the system constructs a base vector through deterministic text-feature mapping and augments it with deliberation metadata, including role, ORID stage, ritual signal, Niche identifier, and turn index. With fixed preprocessing, feature-mapping rules, and auxiliary encodings, identical inputs produce identical representations, which facilitates reproducibility, caching, and comparison across runtime instances.

The deterministic encoder does not depend on an external embedding service and introduces no separately trained embedding model. It is therefore a stable feature encoder rather than a deep semantic encoder; feature collisions and limited invariance to paraphrases or long-range semantic relations remain possible. Role relations, agenda structure, and cross-utterance dependencies are aggregated by subsequent temporal modeling and field queries.

4.4.2. Temporal convolutional sequence modeling

For a deliberation cycle containing n_k temporally ordered utterances, let X denote the encoded utterance sequence.

$$X = [x_1, x_2, \dots, x_{n_k}]^T, \quad x_t \in R^h.$$

Each x_t is the representation of the t -th utterance after text and deliberation-metadata encoding. Temporal modeling preserves order while allowing each position to integrate adjacent responses, cross-round challenges, and cross-agenda relations.

We use an L -layer dilated temporal convolutional network. At layer l , a depthwise convolution extracts temporal relations, a pointwise convolution mixes channels, and layer normalization, nonlinearity, and residual connection update the hidden state as in Eq. (7).

$$\begin{aligned}
Z^{(l)} &= DWConv_{k,\delta_l}(H^{(l-1)}) \\
\tilde{H}^{(l)} &= \phi\left(LN\left(PWConv_l(Z^{(l)})\right)\right) \\
H^{(l)} &= H^{(l-1)} + \tilde{H}^{(l)}, \quad H^{(0)} = X
\end{aligned} \tag{7}$$

Here, k is the convolution-kernel size, δ_l is the dilation factor at layer l , Z_l is the intermediate representation produced by the depthwise convolution, and ϕ is the nonlinear activation. The residual path preserves the previous representation and reduces information loss across layers.

Local dependencies correspond to supplements, agreements, and immediate challenges among adjacent utterances. Longer-range dependencies arise when a tool condition introduced in the factual stage is referenced during solution debate, or when a risk constraint is converted into a rollback rule in the decision stage. Dilated convolution expands the sampling interval without increasing depth linearly with the covered span. The theoretical receptive field R_L is given by Eq. (8).

$$R_L = 1 + (k - 1) \sum_{l=1}^L \delta_l = 1 + (k - 1)(2^L - 1) \tag{8}$$

For a target dependency span, the minimum depth can be selected to satisfy the corresponding receptive-field requirement.

$$L^* = \min\{L: R_L \geq W\}.$$

Equation (8) relates network structure to temporal coverage. Kernel size, depth, and dilation should be configured according to the observed cross-utterance dependency distribution, computational budget, and validation results. The theoretical receptive field describes which utterances can be structurally connected; it does not guarantee that all such context is effectively used.

For computational complexity, a depthwise convolution applies a kernel of length k to h channels, with complexity $O(n_k kh)$; the pointwise convolution performs channel mixing, with complexity $O(n_k h^2)$. For an L -layer depthwise-separable TCN, the overall complexity is given in Eq. (9).

$$O(Ln_k(kh + h^2)) \tag{9}$$

When L , k , and h are fixed, the complexity grows approximately linearly with the number of utterances n_k . This is a structural complexity statement rather than a performance comparison with other sequence models under matched data, hardware, and evaluation settings. Longer sequences can be handled by expanding the effective receptive field, segment-level aggregation, cross-segment state transfer, or long-range attention.

4.4.3. Field queries and candidate provenance aggregation

Different skill fields depend on different evidence. The skill name and description usually combine proposal statements and decision summaries; tools mainly depend on factual-stage capability information; and instructions depend on the action sequence formed during solution debate and decision. We therefore use independent query vectors for different fields rather than generate all fields from one global representation.

Let Q be the set of queried skill fields with $|Q|=q$. For field j , a learnable query vector is matched against the TCN utterance representations. The field-to-utterance relevance weights and aggregated field representation are defined in Eq. (10).

$$s_{j,t} = \frac{j_t^Q W H_t}{\sqrt{h}}, \quad \alpha_{j,t} = \text{softmax}_t(s_{j,t}), \quad F_j = \sum_{t=1}^{n_k} \alpha_{j,t} H_t \tag{10}$$

W is a shared learnable projection matrix, $s_{j,t}$ is the unnormalized relevance score between utterance t and field j , $\alpha_{j,t}$ is the normalized attention weight, and F_j is the field-specific representation.

The system stores one or more focused utterance indices according to the relevance weights and records the original utterance text, agent identifier, role, ORID stage, Niche identifier, and ritual signal. These records form a candidate provenance set for each field and provide input to schema-constrained decoding and the provenance-fidelity gate in Section 5.

The shared projection is computed for all utterances, after which q field queries perform dot products and weighted aggregation over n_k utterances. With fixed q and hidden dimension h , this stage scales approximately linearly with transcript length.

The model is trained with the multi-task objective in Eq. (11).

$$L = L_{dec} + \lambda_{cat}L_{cat} + \lambda_{tool}L_{tool} \quad (11)$$

L_{dec} is the field content generation/reconstruction loss, L_{cat} is the task-category classification loss, and L_{tool} is the multi-label tool-set classification loss. λ_{cat} and λ_{tool} control the contribution of the auxiliary objectives. Category prediction uses a single-label loss, whereas the tool set permits multiple labels.

The query probes are responsible only for core content and capability fields. Preconditions and postconditions are generated by schema-constrained decoding using evidence from factual, risk, and decision stages. Deliberation identifiers, utterance identifiers, and evidence spans are assigned directly from system logs and candidate provenance indices. Version, fitness, and lifecycle state are maintained later by validation and lifecycle-management modules.

4.4.4. Schema-constrained decoding and candidate generation

Field queries and provenance aggregation produce field-specific intermediate representations, which must be converted into a unified, parseable candidate object. Under the skill schema, TSE performs constrained decoding and combines d_k , p_k , and the candidate provenance indices to generate $cand_s^k$, as defined in Eq. (12).

$$cand_s^k = \underset{s \in S(C_{schema})}{argmax} P_\theta(s \mid d_k, p_k) \quad (12)$$

C_{schema} specifies required fields, field types, enumerated values, and hierarchy; $S(C_{schema})$ is the set of candidate structures satisfying those constraints.

Field assignment follows explicit stage boundaries. Core content and capability fields are generated jointly from field-specific representations and the decoder. Preconditions and postconditions are generated using candidate evidence from factual, risk, and decision stages. Deliberation identifiers, utterance identifiers, and evidence spans are written from logs and provenance indices. Version, fitness, state, and lineage are maintained by candidate registration, execution feedback, layered validation, and lifecycle management.

Schema constraints ensure that candidate skills satisfy field, type, and hierarchy requirements. All decoded objects remain candidates and enter the formal skill index only after passing the layered validation and publication criteria in Section 5. Table 1 summarizes skill-definition fields, provenance fields, and lifecycle metadata.

In summary, DART-Net and TSE transform the deliberation state p_k into an intermediate representation d_k and then into a candidate skill state $cand_s^k$. Temporal modeling aggregates information across utterances and agenda stages; field queries construct field-specific representations and candidate provenance indices; and schema-constrained decoding converts those representations into candidate objects. This stage guarantees parseable structure and traceable candidate-source relations, while factual correctness, tool availability, execution safety, and task effectiveness are confirmed by the layered validation and controlled publication process in Section 5.

Table 1: Skill definition, provenance, and lifecycle metadata.

Field group	Field	Meaning
Content	name, description	Skill name and goal description
Capability	category, tools, instructions	Task category, tool set, and execution steps
Constraints	preconditions, postconditions	Applicability conditions, success conditions, and failure boundaries
Provenance	provenance, evidence spans	Deliberation ID, evidence spans, and originating role
Lifecycle	version, fitness, status, lineage	Version, fitness, release state, and parent-child lineage

5. Layered validation and ecological evolution

5.1. Candidate-skill boundary and six-layer gates

DART-Net and TSE convert group deliberation into candidate skill variants with a fixed schema, candidate-source indices, and temporal relations. They perform representation and variation generation rather than safety judgment. Publishability is determined by the subsequent six validation gates.

The six layers are not executed as one simultaneous check. They form a validation chain of pre-check $\rightarrow$ controlled execution $\rightarrow$ regression and publication. A candidate first passes schema, provenance-fidelity, and tool-grounding checks; then it enters a sandbox or controlled canary environment for execution and safety checks; finally, it is evaluated for regression and non-degradation on a common evaluation set. The three-stage gates are summarized in Eq. (13).

$$P_i = G_s G_e G_t, \quad R_i = G_x G_a, \quad L_i = P_i R_i G_r \quad (13)$$

All variables in Eq. (13) are binary, so multiplication is equivalent to logical conjunction. P_i denotes the pre-check gate, R_i the controlled-execution gate, and L_i the full publication gate for candidate i . G_s , G_e , and G_t are the schema, provenance, and tool gates; G_x and G_a are the execution and safety gates; and G_r is the regression/non-degradation gate defined in Section 5.4. $P_i = 1$ allows controlled execution, $R_i = 1$ allows incumbent-challenger comparison, and $L_i = 1$ indicates that the candidate has satisfied the final publication conditions.

5.2. Layered validation and constrained selection

At validation layer m , the system receives candidate c_i , upstream validation results, and the corresponding environment information, and outputs a validation decision, disposition reason, and auditable evidence, as defined in Eq. (14).

$$\gamma_m(c_i) = (b_m, r_m, E_m), \quad m = 1, \dots, 6 \quad (14)$$

The decision is b_m , the reason is r_m , and E_m is the evidence generated or referenced at that layer. Evidence formats may differ—schema-validation logs, source utterance spans, tool metadata, execution traces, safety reports, or regression results—but each evidence object is bound to the candidate identifier, validation layer, timestamp, and environment version.

The aggregate validation state follows the rule reject first, revise second, and compare only after all layers pass, as formalized in Eq. (15).

$$V_i = \text{prio}(b_1, \dots, b_6) \quad (15)$$

The priority in Eq. (15) is reject $>$ revise $>$ pass. If any layer rejects a candidate, the aggregate state is reject. If no layer rejects but at least one requests revision, the aggregate state is revise. Only when all six layers pass does the aggregate state become pass. A pass state only means that the candidate may enter incumbent-challenger comparison; it does not itself mean publication. A revise outcome routes the case back to deliberation supplementation, re-extraction, tool/parameter adjustment, execution-step revision, or rollback-rule supplementation. A reject outcome terminates the candidate path while preserving audit evidence. The candidate becomes published only after the final acceptance condition is satisfied.

5.3. Execution evidence and task-bounded fitness estimation

After the pre-check gate, a candidate is executed in a sandbox, regression suite, or controlled canary environment. The t -th execution of skill i generates the usage record in Eq. (16).

$$\xi_{i,t} = (y_{i,t}, r_{i,t}, l_{i,t}, e_{i,t}, \iota_{i,t}, f_{i,t}, v_{i,t}) \quad (16)$$

In the record, $y_{i,t}$ indicates success, $r_{i,t}$ is normalized task reward, $l_{i,t}$ is latency, $e_{i,t}$ is tool/resource cost, $\iota_{i,t}$ records human intervention, $f_{i,t}$ is a failure-type code, and $v_{i,t}$ is the environment version. Failed executions are retained and contribute zero reward or a task-defined negative penalty rather than being removed from the statistics.

Over a task set D and a fixed observation window, the system aggregates six normalized internal factors: schema/provenance quality, success rate, mean task reward, cross-task or cross-team reuse, execution cost, and sensitivity to environment or tool aging. All factors are normalized to $[0,1]$, with cost and aging sensitivity treated as

negative terms. We interpret the composite F in Eq. (17) as task-bounded fitness, not as universal fitness independent of the task distribution.

$$D_F^i = w^T D_x^i, \quad w \geq 0, \quad 1^T w = 1 \quad (17)$$

The task-bounded fitness $F_i(D)$ is used only for ranking and version competition under the same task distribution, observation window, and normalization convention. Its weights are determined by domain objectives and risk preference and stored with the evaluation configuration.

5.4. Version competition and selection decision

Let $(v)_s^i$ be the incumbent published version and c_i the challenger. Evaluation material consists of a fixed regression set D_{reg} , current task set D_{cur} , historical failure set D_{fail} , and safety counterexample set D_{safe} . D_{reg} and D_{cur} are used for paired utility comparison, while D_{fail} and D_{safe} are checked as hard constraints so that a small number of severe failures cannot be masked by many ordinary successes. Duplicate cases are removed using immutable sample identifiers, and outcomes for each subset are retained separately.

Under the same task, environment version, and random conditions, the challenger and incumbent are evaluated in N paired trials. If $\Delta f_{i,t}$ is the utility contribution difference for trial t , the sample mean is given by Eq. (18).

$$\Delta f_{i,t} = f(c_i; x_t) - f(v_s^i; x_t), \quad \overline{\Delta F_i} = \frac{1}{N} \sum_{t=1}^N \Delta f_{i,t} \quad (18)$$

Because the estimated utility difference is uncertain in a finite sample, let s_Δ be the sample standard deviation of paired differences. We use the one-sided Student-t lower confidence bound in Eq. (19).

$$LCB_{1-\alpha}(\Delta F_i) = \overline{\Delta F_i} - t_{1-\alpha, N-1} \frac{s_{\Delta, i}}{\sqrt{N}} \quad (19)$$

If the empirical distribution is clearly inconsistent with the parametric assumptions, a paired bootstrap lower bound can be used instead. The publication threshold δ_{min} is set to 0 for strict non-degradation or to a positive value when a minimum improvement is required.

Key metrics are constrained according to direction. Metrics for which larger is better must satisfy their lower bounds, whereas metrics for which smaller is better must satisfy their upper bounds. Safety violations, unauthorized actions, and severe failures are zero-tolerance events; positive safety metrics such as safety pass rate must meet preset lower bounds. Let C_i indicate that all metric constraints hold and H_i indicate that all safety counterexamples pass and no historical severe failure reappears. The regression gate and final acceptance condition are defined in Eq. (20).

$$G_r = 1[LCB_i \geq \delta_{min}] C_i H_i, \quad a_i = P_i R_i G_r \quad (20)$$

In Eq. (20), the bold $1[\cdot]$ is an indicator function. Because P_i , R_i , G_r , C_i , and H_i are binary, multiplication is equivalent to logical conjunction. If $a_i = 1$, a new version is generated and marked published while the incumbent is retained as a historical rollback point. Otherwise, the incumbent remains active and the challenger, failed gates, evaluation samples, and disposition reason are recorded. Thus, non-degradation is enforced conditionally on the specified evaluation set, confidence level, metric tolerances, and environment version.

5.5. Skill variation and renewed deliberation

Skill variation consists of candidate formation, pre-validation, controlled execution, usage-record aggregation, version competition, and a selection decision. Publication retains a new version; revision turns failure evidence into conditions for the next variation; rejection preserves audit evidence and terminates that variant. Failure types route separately to further deliberation, re-extraction, tool or parameter adjustment, execution-step revision, or risk memory.

Execution traces, failure types, environment changes, and human interventions enter the four-layer memory core and influence factual supplementation, risk challenges, tool selection, and candidate revision in the next round. The system thereby closes the loop deliberation $\rightarrow$ extraction $\rightarrow$ validation and publication $\rightarrow$ assignment and execution $\rightarrow$ usage feedback $\rightarrow$ skill update $\rightarrow$ memory update $\rightarrow$ renewed deliberation, rather than treating validation failure as a terminal dead end.

5.6. Lifecycle states and retention routing

The skill lifecycle contains draft, revision, ready, published, degraded, and deprecated states. Constraint supplementation, tool replacement, workflow composition, and subskill decomposition create challenger versions instead of overwriting the current published version. A published skill may move to degraded or deprecated after sustained low utility, environmental incompatibility, or long inactivity, while historical versions and their validation evidence are retained.

To suppress short-term oscillation, lifecycle transitions use a three-cycle majority rule: a promotion or demotion is applied only when the same condition is satisfied in at least two of the latest three evaluation cycles. Safety violations and severe failures bypass this debounce rule and can immediately block or degrade a skill.

SkillRouter first recalls candidates using Okapi BM25 [24], TF-IDF, character n-grams, and synonym expansion, then reranks them jointly using name, description, instructions, category, and tools. Production routing selects only published skills. Ready, revision, and draft skills are restricted to sandbox, regression, or challenger workflows; degraded skills are normally excluded from new production tasks but retained for compatibility, diagnosis, and rollback; deprecated skills are excluded from routine retrieval. Routing logs record the query, candidate versions, state filters, final scores, and selection reasons.

5.7. Skill evolution under ecological selection

Ecological selection is not an additional stage placed after validation; it is the underlying regularity that determines what counts as evolution in this framework. A cycle is evolutionary only when skill variants persist differentially under task and environmental pressure and the retained result enters a successor state. Plaza and DART-Net/TSE increase the rate of useful variation and shorten unguided search; they do not replace environmental selection.

Section 5 implements two levels of selection. Human-constrained pre-selection uses the six gates, controlled execution, and version competition to remove variants with incomplete schema, weak provenance, unavailable tools, failed execution, safety violations, or clear degradation. Ecological environmental selection then uses resource abundance, predation pressure, and niche capacity to create differential persistence among published skills and combinations. Passing the gates does not prove environmental adaptation, and ecological frequency does not directly publish a version. Persistent combinations, uncovered demand, and recurrent failures return as evidence for a challenger or renewed deliberation.

Section 5 therefore engineers the ecological selection sequence deliberation-accelerated variation, constrained pre-selection, environmental selection, and differential retention. Chapter 6 transfers published versions, execution outcomes, validity conditions, failure boundaries, and provenance so that capability surviving selection can persist. The present ecological experiment tests whether this chain operates and yields initial persistence signals; it does not support universal claims of causality, optimality, or convergence.

Figure 5 presents the engineering pre-selection and version-governance sub-flow within ecological evolution. Differential-persistence evidence from environmental selection returns through the usage-feedback entry.

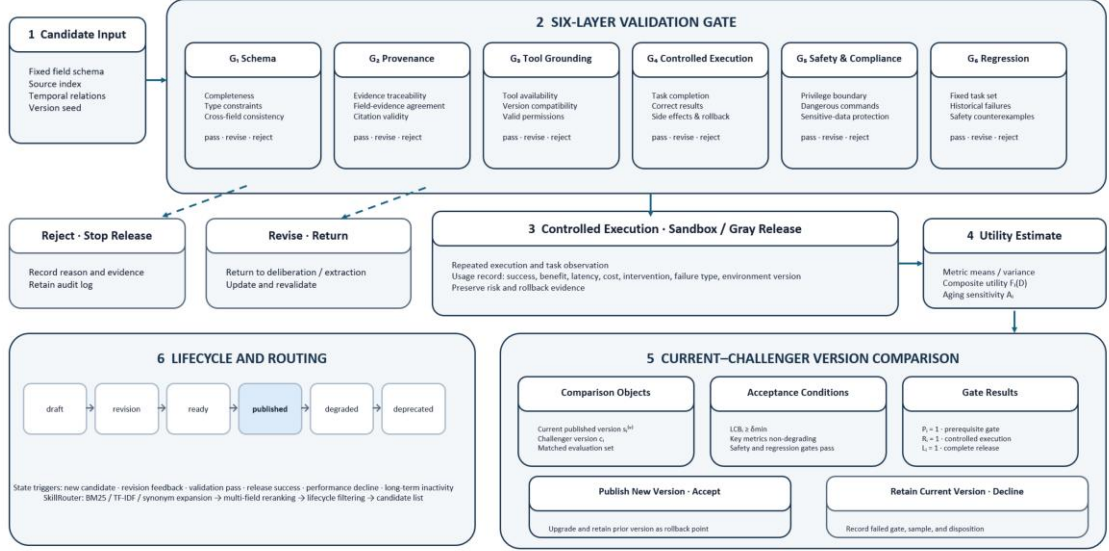


Figure 5: Layered pre-selection, version competition, and skill retention within ecological evolution.

6. Agent memory core and post-selection inheritance

6.1. Inheritance goals and selection boundary

Memory inheritance is the retention stage of skill evolution. Its purpose is not to copy all history, but to transfer published skill versions together with their provenance, execution outcomes, validity conditions, and risk boundaries during agent upgrade, replacement, or cross-team transfer. Still-valid intentions and necessary failure records are also preserved so that successors can distinguish selected capabilities, unverified candidates, and discarded paths.

Inheritance can be evaluated at three levels. Migration integrity checks whether the data, schema, and provenance identifiers selected by the Will declaration arrive at the successor. Protocol robustness tests whether contamination, version conflict, transfer faults, or write failures can be isolated, rejected, or rolled back. Behavioral adaptation compares inherited and cold-start agents in task success, adaptation rounds, repeated historical failures, and negative transfer. The experiments in this paper focus on the first two levels; behavioral adaptation is reserved for controlled inherited-vs-cold-start comparisons in future work.

6.2. Four-layer parallel memory state and update

Let i denote the agent and t the memory-update cycle. Each agent maintains the four-layer state in Eq. (21).

$$m_{i,t} = (E_m^{i,t}, P_m^{i,t}, I_m^{i,t}, A_m^{i,t}) \quad (21)$$

Event memory stores model calls, tool executions, and environment interactions permitted by access-control, privacy, and retention policies. Each record preserves task, time, and provenance information for traceability, failure analysis, and audit.

The perception stream stores recent environmental and group observations in a capacity-bounded temporal window. Expired, duplicated, or out-of-scope perceptions are removed according to policy.

The intention queue manages intentions that have not been sent, acknowledged, or completed and records their goals, dependencies, deadlines, responsible entity, and completion state, maintaining goal continuity across multi-turn tasks.

Risk-tendency residue represents post-failure caution and decision preference with low-dimensional variables such as risk label, intensity, valence, and arousal. These variables describe machine decision tendencies and do not imply human-like subjective emotion.

The deliberation state p_k in Section 4 is a group-level state. Each agent receives only the deliberation view relevant to its role, utterances, Niche functions, and task relations; denote this view $\Pi_i(p_k)$.

$$view_p^{i,t} = \Pi_i(p_t)$$

The deliberation view and the execution evidence defined in Section 5.3 jointly drive the update of each memory layer, as represented in Eq. (22).

$$x_m^{i,t+1} = C_x(x_m^{i,t}, \Pi_i(p_t), \xi_{i,t}), \quad x \in \{E, P, I, A\} \quad (22)$$

The four operators respectively perform event append/provenance association, perception write/window truncation, intention progression/termination, and risk update/decay. They share the same deliberation view and execution evidence but apply layer-specific retention, overwrite, and expiration rules.

Risk-tendency decay can be written in a generic exponential form.

$$a_{i,t+\Delta t} = a_{i,t} 2^{-\Delta t/T_{1/2}}$$

The half-life is configurable. Perception-window capacity, risk-decay parameters, cleanup periods, and concrete data structures are implementation and experimental settings reported in Section 7.

6.3. SWEI controlled inheritance protocol

When an agent retires, is upgraded, or migrates, SWEI executes four stages. Seal freezes source memory and creates a read-only snapshot. Will specifies the source and beneficiary agents, transferable memory scope, retention period, permission boundaries, and conflict policy. Export creates a transfer package carrying provenance and verification information. Import validates the package in quarantine before activation, rejection, or rollback.

SWEI transfers only content authorized by the Will declaration and accepted by the import gates. Success evidence for published skills supports capability continuity, while severe failures, safety violations, and rollback reasons are retained to prevent successors from repeating paths already removed by selection.

Different inheritance strategies can be assigned to the four memory layers, as represented by Eq. (23).

$$\pi_{i \rightarrow j,t} = (\pi^E, \pi^P, \pi^I, \pi^A)_{i \rightarrow j,t} \quad (23)$$

The four strategy components apply to event, perception, intention, and risk-tendency partitions and each takes replace, merge, or selective. Replace substitutes the Will-selected partition only after preserving a pre-import snapshot; merge combines records using immutable identifiers, time, and provenance rules; selective imports only explicitly selected records or fields. This permits event logs to be merged while stale perceptions, unfinished intentions, and risk tendencies can be handled more conservatively.

Let i be the source agent and j the successor. The transfer package is defined in Eq. (24).

$$z_{i \rightarrow j,t} = \text{Export}(\text{Seal}(m_{i,t}), W_{i \rightarrow j,t}) \quad (24)$$

$W_{i \rightarrow j,t}$ is the inheritance declaration specifying beneficiary, layer scope, strategy vector, permissions, and conflict rules; $z_{i \rightarrow j,t}$ is the package exported from the read-only snapshot. Concrete serialization formats, fields, and verification implementation are experimental settings.

Before activation, an imported package must pass scope, schema, integrity, provenance/permission, and conflict-processability gates, summarized in Eq. (25).

$$g_{imp} = \bigwedge_{v \in V_{imp}} g_v \quad (25)$$

Let V_{imp} be the set of import checks. If any component gate is zero, the aggregate gate g_{imp} is zero. The import transaction is then defined in Eq. (26).

$$m_{new}^j(j, t) = T_\pi(m_{old}^j(j, t), z_{i \rightarrow j,t}, g_{imp}) \quad (26)$$

$old_m^{j,t}$ and $new_m^{j,t}$ are the successor memory states before and after the transaction, and T_π is the gated transaction operator. When $g_{imp} = 1$, the package is applied according to the layer-specific strategy; otherwise the pre-import state is returned. If any check, conflict-resolution step, or write operation fails, the transaction result remains the pre-import state. Merge and selective do not overwrite existing successor records directly, and replace also requires a recoverable snapshot.

The protocol therefore provides constrained state transfer, provenance retention, and failure atomicity, as illustrated in Fig. 6.

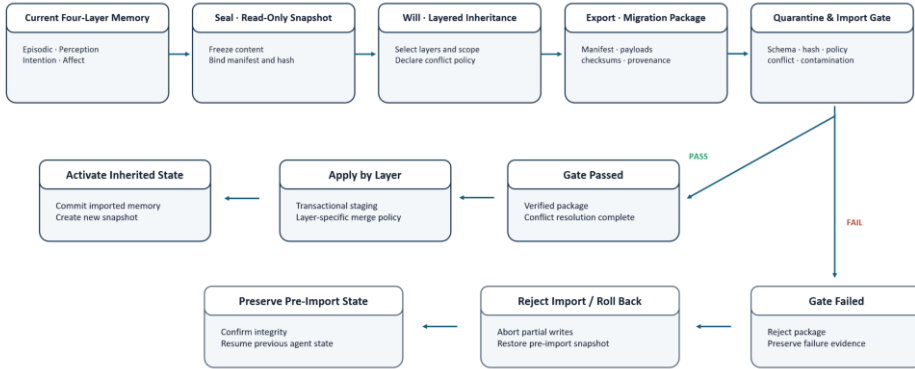


Figure 6: Seal, layered inheritance, import gating, and transactional rollback of agent memory.

7. Experiments and results

Experiments follow the engineered ecological sequence. Sections 7.2-7.6 test whether deliberation and extraction accelerate the production of traceable variants. Sections 7.7-7.9 test lifecycle management, routing, and closed-loop updating as constrained pre-selection. Section 7.10 directly observes differential persistence under ecological pressure and a multigenerational run. Sections 7.11-7.14 test whether environmentally selected capability and its evidence can be inherited, retained, and recovered safely. The ecological experiment is not an appendix; it operationalizes the underlying selection mechanism. Its limited sample provides revision evidence but cannot by itself determine publication or establish a universal law.

7.1. Experimental settings and metrics

Cluster operations are used as the validation domain. The experiments cover Plaza deliberation, DART-Net/TSE candidate extraction, four-layer memory inheritance, lifecycle classification, SkillRouter retrieval, short closed-loop evolution, memory consolidation and controlled forgetting, and robustness of the transfer/safety chain. Metrics include candidate count, field completeness, annotation agreement, role/provenance structure, processing latency, migration integrity, retrieval hits, internal quality, and abnormal-recovery outcomes.

Candidate extraction uses 256-dimensional base utterance representations, a three-layer TCN, kernel size 3, and dilation sequence [1, 2, 4]. Field-to-utterance alignment covers five core fields: name, description, category, tools, and instructions. Memory experiments use the four-layer event/perception/intention/risk state, and skill-governance experiments follow the publication gates, lifecycle rules, and feedback-update mechanism.

7.2. Effect of the ORID agenda on skill-variation formation

This experiment asks whether the ORID agenda improves the extractability of candidate skills from group deliberation. In one operations-optimization topic, five agents produced 28 utterances over four rounds, including three CHALLENGE and two AGREE signals. Four valid five-finger scores were all at least 4, with a mean of 4.25 and no blocking score of 1. Fact statements, risk challenges, proposal support, and consensus closure were therefore all activated within one deliberation process, leaving a continuous record for extraction.

We further compare three conditions—complete ORID, ORID removed, and free chat—using candidate count, field completeness, and annotation agreement as output-quality measures (Table 2 and Fig. 7). Complete ORID yields

2.00 candidate skills per discussion, 91% field completeness, and $\kappa=0.87$. Removing ORID reduces the three measures to 0.75, 54%, and 0.51; free chat further reduces them to 0.50, 33%, and 0.19.

Relative to complete ORID, removing ORID reduces candidate count by 62.5%, lowers field completeness by 37 percentage points, and reduces κ by 0.36. Free chat reduces candidate count by 75%, field completeness by 58 percentage points, and κ by 0.68. Candidate yield, field completeness, and annotation agreement therefore decline in the same direction as agenda structure is weakened.

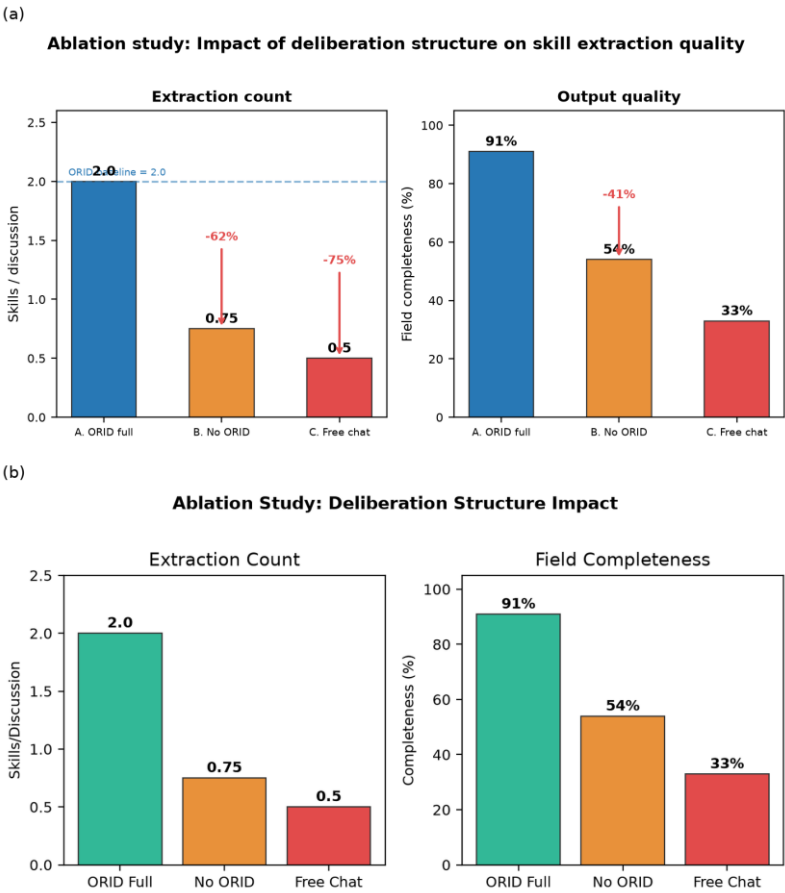


Figure 7: Effect of ORID agenda structure on candidate-skill output quality: (a) candidate count and field completeness; (b) candidate-output quality comparison.

The results indicate that ORID organizes facts, risks, solution interpretations, and decisions into a more complete deliberation record. Candidate skills are not only more numerous but also have more complete fields and more stable annotation boundaries. Complete ORID therefore improves candidate-skill extractability under the evaluated conditions.

Table 2: Effect of ORID agenda structure on candidate-skill output quality.

Condition	Skills / discussion	Field completeness	Kappa
Complete group deliberation	2.00	91%	0.87
ORID removed	0.75	54%	0.51
Free chat	0.50	33%	0.19

7.3. Plaza components as sources of skill variation

To distinguish the roles of the ORID agenda, Niche role-function positions, and ritual signals, we compare complete Plaza, Plaza without ORID, without Niche, without ritual signals, and free chat. Candidate count and field completeness measure output quality, while role coverage, cross-role CHALLENGE, provenance-role entropy, and signal distinguishability characterize the deliberation process (Table 3).

Under complete Plaza, candidate count and field completeness are 2.00 and 91%. Removing ORID lowers them to 0.75 and 54%; free chat lowers them further to 0.50 and 33%, showing that ORID provides the main mechanism for consolidating dispersed utterances into extractable candidate knowledge. Removing Niche leaves candidate count and completeness at 2.00 and 91% but affects role coverage, cross-role CHALLENGE, and provenance-role entropy, indicating that Niche primarily supports knowledge-function coverage, provenance diversity, and cross-role checking. Removing ritual signals leaves candidate count and completeness unchanged but makes CHALLENGE and AGREE indistinguishable as independent traceable log events, showing that ritual signals mainly improve process identifiability and traceability.

The five conditions reveal complementary roles: ORID drives candidate formation and field consolidation; Niche maintains knowledge-function coverage, provenance diversity, and cross-role checking; and ritual signals make objection, agreement, and consensus processes explicitly traceable. Plaza therefore combines agenda organization, role topology, and deliberative signals rather than relying on a single mechanism.

Table 3: Candidate outputs and process metrics under different Plaza configurations.

Condition	Skill count	Completeness	Structural change
A Complete Plaza	2.00	91%	Role coverage, signals, and provenance retained
B Without ORID	0.75	54%	Information partitioning and agenda progression absent
C Without Niche	2.00	91%	Role coverage=0; cross-role challenge=0; entropy=0.078
D Without signals	2.00	91%	CHALLENGE/AGREE no longer distinguishable
E Free chat	0.50	33%	Agenda, roles, signals, and facilitation absent

7.4. Structural completeness of deliberative variation

Under the complete Plaza configuration, we compute seven process metrics over five discussions across six dimensions: role configuration, role activation, cross-role interaction, risk boundaries, provenance diversity, and tool-information coverage (Table 4 and Fig. 8). C_{role} measures the match between task-required knowledge functions and configured roles, whereas role coverage measures whether those functions are actually activated in the discussion.

C_{role} and role coverage are both 1.0, and the cross-role CHALLENGE ratio is also 1.0, showing that all required knowledge functions are configured and activated and every recorded challenge crosses a role or knowledge-function boundary. Provenance-role entropy is 0.922 and the mean single-role dominance ratio is 0.40, indicating contributions from multiple roles without absolute dominance by one source. Risk-boundary coverage is 0.48 ± 0.24 and the tool-mention rate is 0.31 ± 0.07 , showing that risks, failure conditions, rollback boundaries, and tool/pipeline information enter the deliberation record and can support constraint and tool fields in candidate skills.

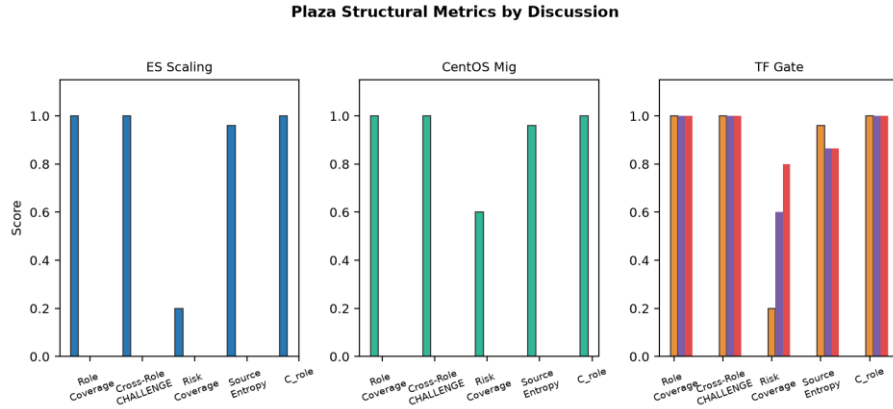
The complete Plaza configuration therefore activates knowledge functions, supports cross-role checking and multi-source participation, and records risk boundaries and tool information in computable, traceable form.

Table 4: Structural completeness metrics of Plaza deliberation across five discussions.

Metric	Mean / result	Interpretation
C_role	1.00	Required and participating roles match
Role coverage	1.00	100% of required domain roles activated
Cross-role CHALLENGE	1.00	All recorded challenges cross role boundaries
Risk-boundary coverage	0.48 ± 0.24	Risks, failures, and rollback boundaries explicitly quantified
Provenance-role entropy	0.922	Candidate provenance remains highly diverse

Metric	Mean / result	Interpretation
Single-role dominance	0.40	No single role dominates on average
Tool mention rate	0.31 ± 0.07	Tool and execution-pipeline information consistently enters deliberation

(a)



(b)

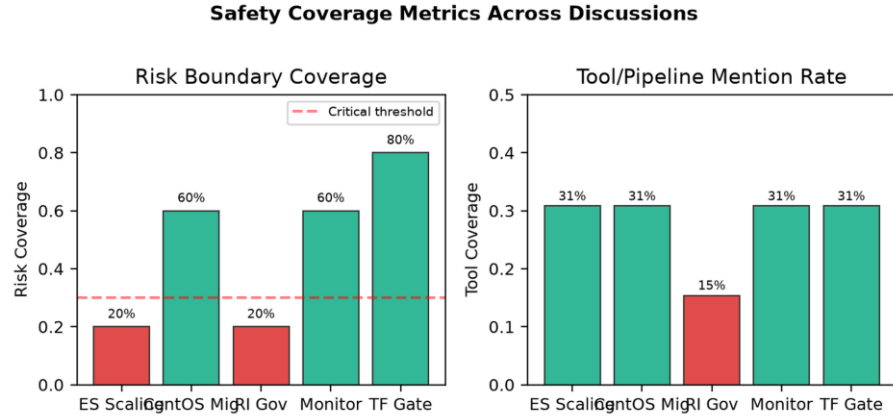


Figure 8: Structural completeness of Plaza deliberation: (a) role and provenance structure; (b) risk-boundary and tool-information coverage.

7.5. Local candidate-extraction latency and stage cost

This experiment tests whether DART-Net/TSE can convert Plaza transcripts into candidate skills with low and stable local computational overhead. Five transcripts, each containing five utterances, are run independently ten times. The timed pipeline includes utterance encoding, TCN temporal modeling, field-query aggregation, and local candidate synthesis (Table 5 and Fig. 9).

The five mean latencies are 4.83, 4.70, 4.82, 4.81, and 4.89 ms, with an overall mean of 4.81 ± 0.06 ms. The gap between maximum and minimum means is 0.19 ms, only 3.95% of the overall mean, indicating little topic-dependent latency variation at fixed input size. TCN temporal modeling accounts for approximately 60% of the total, or about 2.9 ms at the overall mean, and its three-layer receptive field covers the five input utterances used in this experiment.

DART-Net/TSE therefore completes utterance encoding, temporal modeling, field aggregation, and candidate synthesis in approximately 4.81 ms while maintaining stable millisecond-scale processing across the tested discussions.

Table 5: Local candidate-skill extraction latency of the DART-Net/TSE pipeline (10 runs per transcript).

Discussion topic	Utterances	Focused utterances	Local extraction latency (ms)
ES Instances Scaling	5	5	4.83

Discussion topic	Utterances	Focused utterances	Local extraction latency (ms)
CentOS-Rocky Migration	5	5	4.70
Resource Cost Governance	5	5	4.82
Monitoring Rollback	5	5	4.81
Terraform Change Gate	5	5	4.89

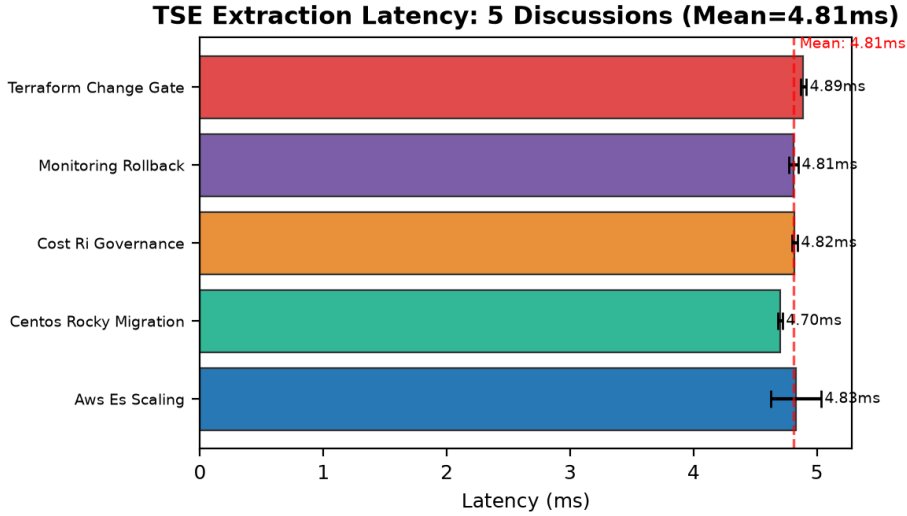


Figure 9: Local DART-Net/TSE candidate-skill extraction latency on five discussions (mean 4.81 ms; error bars show standard deviation over 10 runs).

7.6. Evidence alignment for candidate variations

This analysis asks whether the five candidate-skill fields—name, description, category, tools, and instructions—can be associated with supporting utterances in Plaza transcripts. Figure 10 uses field-specific keyword seeds to construct a query-attention heatmap over 12 diagnostic samples comprising 53 utterances, providing a diagnostic view of evidence association and provenance traceability.

Field-to-utterance weights range from 0.075 to 0.435, and the ratio of each field's maximum weight to its mean ranges from 1.79 to 2.07. Local high-weight regions show field-specific associations: tool names, commands, and interfaces mainly support tools; action verbs and step relations mainly support instructions; and problem framing and proposal summaries support name, description, and category.

The normalized entropy in Fig. 10 ranges from 0.981 to 0.990, indicating that evidence for a field is often distributed across multiple relevant utterances rather than forming a strict one-utterance/one-field mapping. The results show that candidate fields can be linked back to supporting deliberation utterances, providing basic provenance explanation and a continuous chain from traceable source to inspectable candidate and controlled publication.

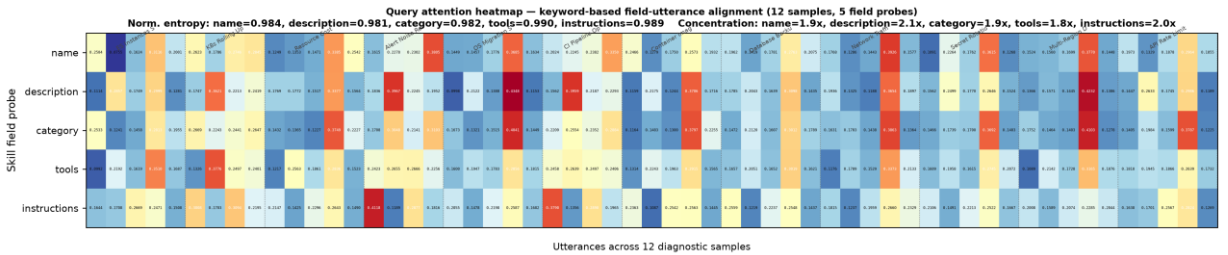


Figure 10: Query attention heatmap—keyword-based field-to-utterance alignment.

7.7. Lifecycle selection of skills

This experiment tests whether lifecycle classification with three-cycle debouncing can stably assign skills to the general or reserve pool. Classification combines reuse scope, publication gates, usage evidence, internal utility, and environment state. A pool-state change is confirmed only when the same promotion or demotion decision appears in at least two of the most recent three evaluation cycles (Table 6 and Fig. 11).

None of the five skills oscillates during the three cycles. CentOS-Rocky Migration and Resource Cost Advisor have cross-task adoption records and pass publication gates, so they enter the general pool. ES Instances Scaling remains in reserve because its usage evidence is still insufficient. Old Monitoring Setup has effectiveness 0.28, below the 0.4 threshold, and is not promoted. Legacy Terraform 0.12 is returned to reserve because its lifecycle state is degraded.

Two skills enter general and three remain in reserve, giving a 40% generalization ratio. The results show stable governance in which skills are promoted when evidence is sufficient, returned when performance degrades, and protected from repeated switching caused by short-term fluctuations.

Table 6: Skill lifecycle classification results.

Skill	Instant class	After three cycles	Rationale
ES Instances Scaling	reserve	reserve	No usage record
CentOS-Rocky Migration	general	general	Adopted by 2 teams; publication gate passed
Resource Cost Advisor	general	general	Adopted by 2 teams; publication gate passed
Old Monitoring Setup	reserve	reserve	effectiveness=0.28<0.4
Legacy Terraform 0.12	reserve	reserve	lifecycle=degraded

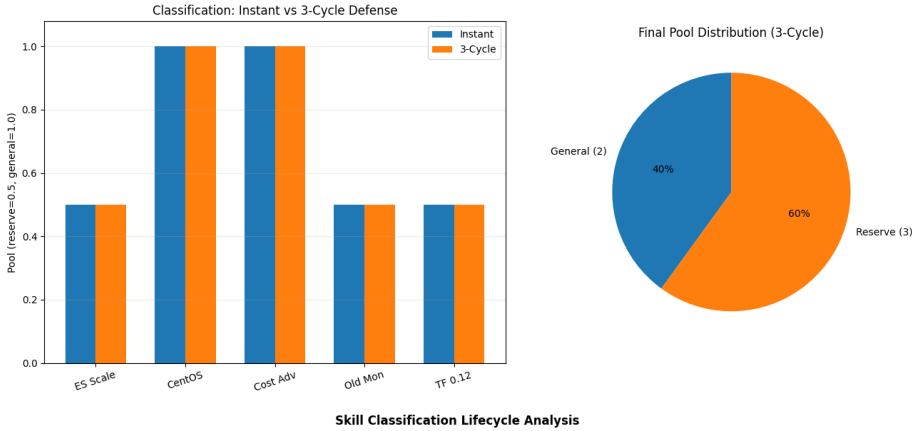


Figure 11: Instant classification, three-cycle debouncing, and final skill-pool distribution.

7.8. Skill routing under selection constraints

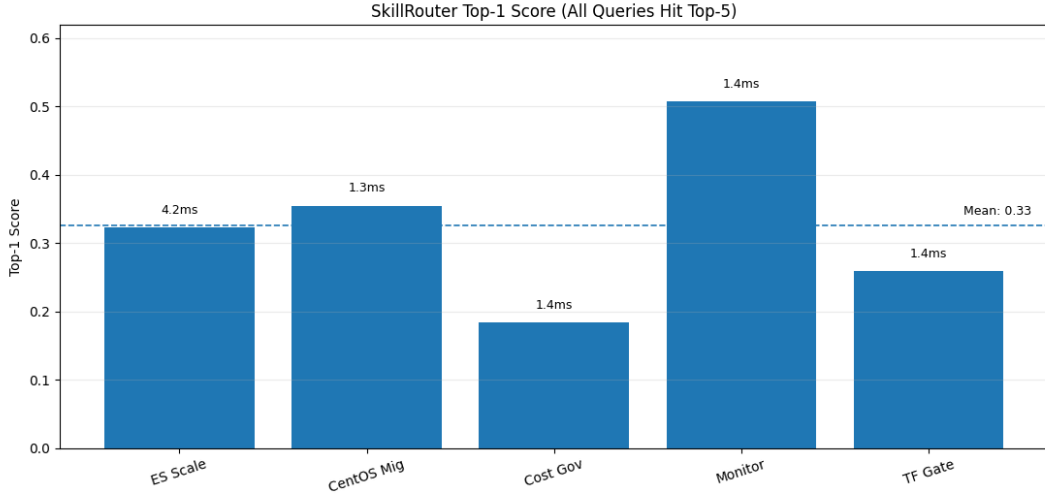
This experiment tests whether SkillRouter can rapidly recall candidate skills matching task semantics, applicability conditions, and tool requirements as the skill pool grows. Five queries are evaluated against a simulated pool of 30 skills using two-stage recall and reranking (Table 7 and Fig. 12).

Top-1 scores across the five queries range from 0.184 to 0.507, indicating differentiated matching strength. Mean processing latency is 1.94 ms. The expected skill appears in the Top-5 set for all five queries, yielding a 100% Top-5 hit rate. Resource Cost Governance has a Top-1 score of only 0.184 but remains within Top-5, illustrating that retaining a candidate set reduces miss risk when the highest individual score is low.

Under the current pool size, SkillRouter therefore narrows the candidate range at low latency and reliably forwards the expected skill to subsequent state checking, constraint matching, and invocation gating.

Table 7: SkillRouter retrieval results.

Query	Top-1 result	Category	Score	Top-5 hit	Latency (ms)
ES instance scaling	ES Instances Scaling	automation	0.323	Yes	4.2
CentOS-to-Rocky migration	CentOS-Rocky Migration	automation	0.355	Yes	1.3
Procurement and cost optimization	Resource Cost Governance	domain knowledge	0.184	Yes	1.4
Monitoring alert rollback	Monitoring Rollback	monitoring	0.507	Yes	1.4
Terraform safety check	Terraform Change Gate	automation	0.259	Yes	1.4

*Figure 12: Top-1 score distribution for five SkillRouter queries.*

7.9. Short-horizon skill-evolution trajectory

This experiment tests whether deliberative discovery $\rightarrow$ candidate validation $\rightarrow$ controlled publication $\rightarrow$ task use $\rightarrow$ feedback-driven updating forms a continuous loop. Across four cycles, we track the number of skills and the internal quality score Q . Q is the weighted aggregate of the six normalized internal factors in Eq. (17), scaled to $[0,1]$. Table 8 and Fig. 13 report the results.

With agenda depth ≈ 0.8 , memory-inheritance ratio ≈ 1.0 , and execution feedback enabled, skill count evolves from 5 to 7, 9, and 9, while Q increases from 0.42 to 0.63, 0.78, and 0.78. Relative to the initial state, the final skill count is 80% higher and Q is approximately 85.7% higher, with no decrease in Q while the pool expands.

During the first two update cycles, skill count and Q increase together, indicating that newly formed candidates enter the pool after validation/publication while execution feedback improves quality. In the final cycle both metrics remain at 9 and 0.78, showing that when no new effective evidence is available, publication gating can stop further expansion while maintaining the achieved quality level.

The short-run experiment therefore demonstrates continuous data return across candidate discovery, validation/publication, task use, and feedback updating, combining skill growth and internal quality improvement with suppression of unsupported expansion.

Table 8: Short-run changes in skill count and internal quality.

Iteration	Skill count	Δ skills	Quality Q	ΔQ
0	5	—	0.42	—
1	7	+2	0.63	+0.21
2	9	+2	0.78	+0.15
3	9	0	0.78	0

Closed-Loop Convergence Trajectory

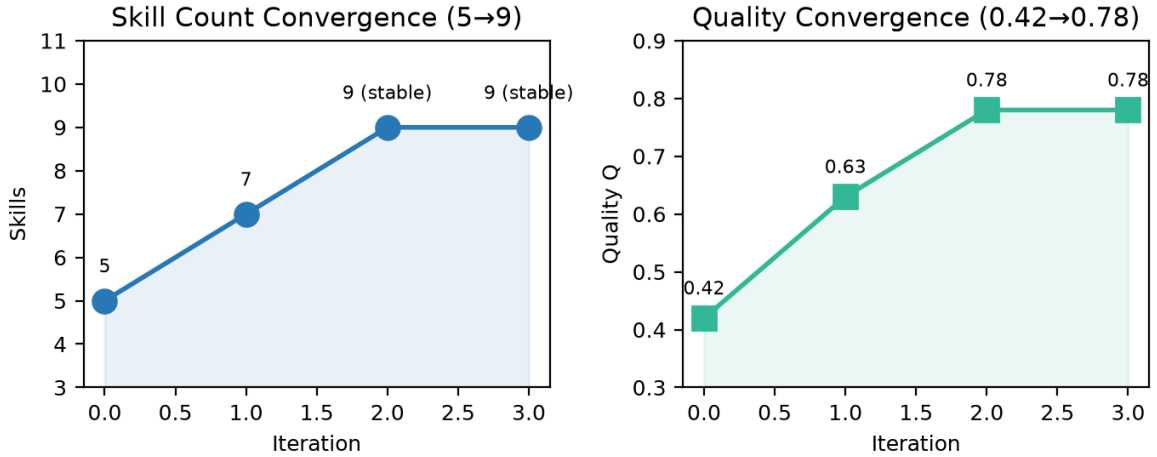


Figure 13: Short-run trajectory of skill count and internal quality score.

7.10. Ecological selection and multigenerational persistence

This experiment corresponds to environmental selection in Section 5.7. It does not use a limited run to ask whether natural selection exists; it asks whether the framework can operationalize that regularity by imposing resource, predation, and niche pressure and recording differential persistence in published-skill use, collaboration behavior, and skill combinations. Four resource-abundance settings $A \in \{0.35, 0.45, 0.55, 1.40\}$ are used. Each environment contains five agents and runs mainly for one to two generations; one mixed run extends to nine generations. The experiment records resource abundance, predation pressure, niche capacity, skill configurations, behavioral events, and survival timelines. Table 9 and Fig. 14 report the results. Because independent replication is limited, the findings describe only the current sample and cannot be generalized as an ecological law.

Behavioral events are partitioned into three mutually exclusive classes: collaboration behavior, defined as receiving shared resources or successfully following a signal; independent skill use, defined as skill-matched foraging when no collaboration event is recorded; and other behavior. The three frequencies sum to one. This classification describes behavioral composition and does not establish a causal contribution to survival.

Table 9: Survival time and behavior frequencies under different ecological-pressure conditions.

Regime	Resource abundance A	Best survival time	Mean collaboration frequency	Mean independent skill-use frequency
Abundant	1.40	86	4.9%	2.8%
Pressure	0.55	67	13.2%	7.9%
Scarce	0.45	61	13.6%	3.7%
Harsh	0.35	60	2.5%	4.8%

Note: The data come from an exploratory run under one parameter configuration and have not yet been validated through multiple independent repetitions.

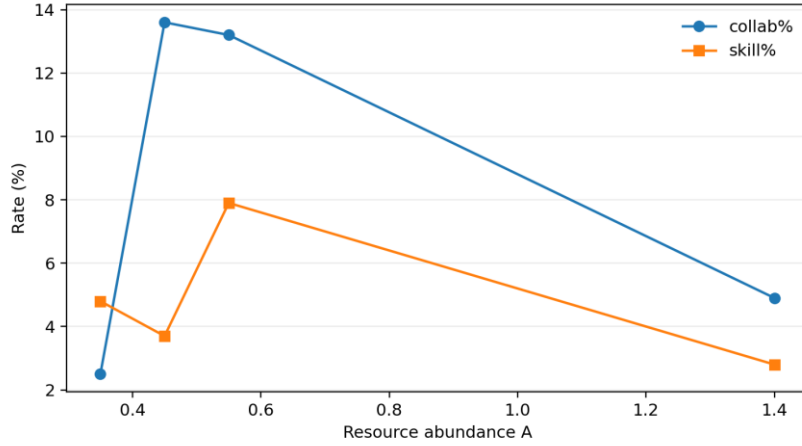


Figure 14: Collaboration and independent skill-use frequencies across resource-abundance regimes.

Table 9 and Fig. 14 show collaboration frequencies of 2.5% in the harsh regime, 13.6% in the scarce regime, 13.2% in the pressure regime, and 4.9% in the abundant regime. A sample-level quadratic fit yields an inverted-U pattern (quadratic coefficient -0.639 , $p=0.004$, $R^2=0.43$), whereas independent skill use does not show the same structure ($R^2=0.04$). The p value describes only this sample fit; without multiple independent repetitions and a permutation test, it does not establish reproducibility or generalizability. The single nine-generation mixed run produces a non-empty persistent skill combination, but generational depth and recombination change together, so this is persistence evidence rather than a causal effect. These results can inform the next variation and version competition; they do not establish causal direction or directly determine publication.

7.11. Post-selection memory inheritance for a single agent

This experiment tests whether the schema and record counts of the four memory layers—event log, perception stream, intention queue, and risk tendency—remain intact when transferred through SWEI during agent upgrade or replacement. After three task cycles, the source agent contains 847 event records, 312 perception records, 3 unfinished intentions, and 12 risk-tendency records. The system performs Seal, Will, Export, and Import and records package size, import latency, schema checks, and record-count checks (Table 10).

To measure migration integrity under the scope selected by the Will declaration, we define the record-count integrity ratio in Eq. (27).

$$R_{count} = \frac{N_{imp}}{N_{sel}} \quad (27)$$

N_{sel} is the total number of records selected for transfer by the Will declaration and N_{imp} is the number successfully written into successor memory and passing the count check. $R_{count} \in [0,1]$ is calculated separately for event, perception, intention, and risk-tendency partitions.

The migration selects 1174 records. The export package is 1.2 MB and import latency is 2.3 ms. After import, the four layer counts remain 847, 312, 3, and 12. All schema checks pass, and both per-layer and overall count integrity equal 1.00.

The SWEI protocol therefore preserves the tested schema and record-count integrity during single-agent upgrade or replacement while restoring the selected memory to the successor with low latency.

Table 10: Data-schema and record-count integrity of single-agent memory export and import.

Memory layer	Source count	Inherited count	Count integrity
Event log	847	847	100%
Perception stream	312	312	100%
Intention queue	3	3	100%

Memory layer	Source count	Inherited count	Count integrity
Risk-tendency labels	12	12	100%

7.12. Multi-agent and cross-team capability inheritance

This experiment tests whether SWEI can extend from a single migration to multiple agent pairs and cross-team handover. Three source-successor pairs execute the complete Export-Import path. Package sizes range from 63.1 to 71.3 KB with a mean of 65.9 KB (Table 11 and Fig. 15). The range is 8.2 KB, approximately 12.4% of the mean. Event-log and perception-stream record-count integrity is 100% for all three pairs, indicating that differences in source agent and memory load do not break migration of the principal partitions.

For cross-team transfer, operations (ops) expert memory is imported into security and FinOps successor agents. Both imports pass receiver-side schema validation (Fig. 16). The reported 20 records refer to ops-related events, whereas 40 refers to the total inherited records. In both transfers, provenance relations, beneficiary identity, and inheritance scope are carried and recognized by the receiver.

SWEI thus extends from single-agent migration to multi-instance and cross-team handover while preserving the tested record counts and transfer-package provenance relations.

Table 11: Results of multi-agent memory inheritance and cross-team transfer.

Experiment	Object	Result	Interpretation
Three agent-pair transfers	3 source/successor pairs	63.1–71.3 KB export; event/perception integrity 100%	All three transfers preserve 100% event/perception record counts
ops → security	1 cross-team import	20 ops-related events; 40 inherited records; schema check passed	Transfer package recognized and imported by security successor
ops → finops	1 cross-team import	20 ops-related events; 40 inherited records; schema check passed	Transfer package recognized and imported by FinOps successor

Multi-Agent Memory Inheritance: 3 Pairs, Cross-Team Verified

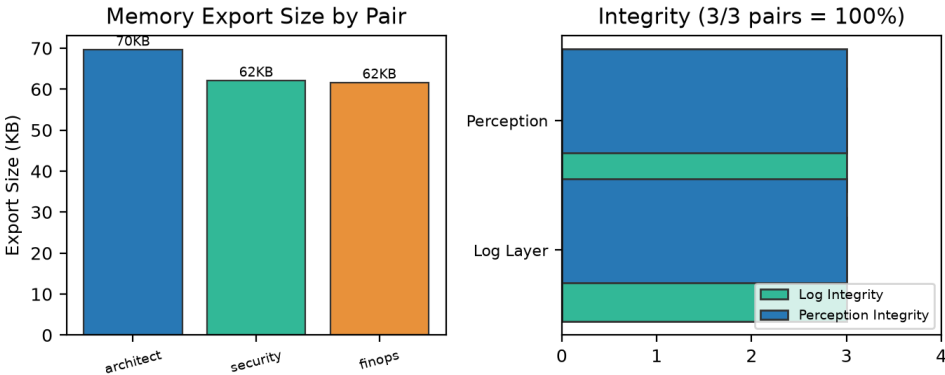


Figure 15: Memory export size and integrity for three agent pairs.

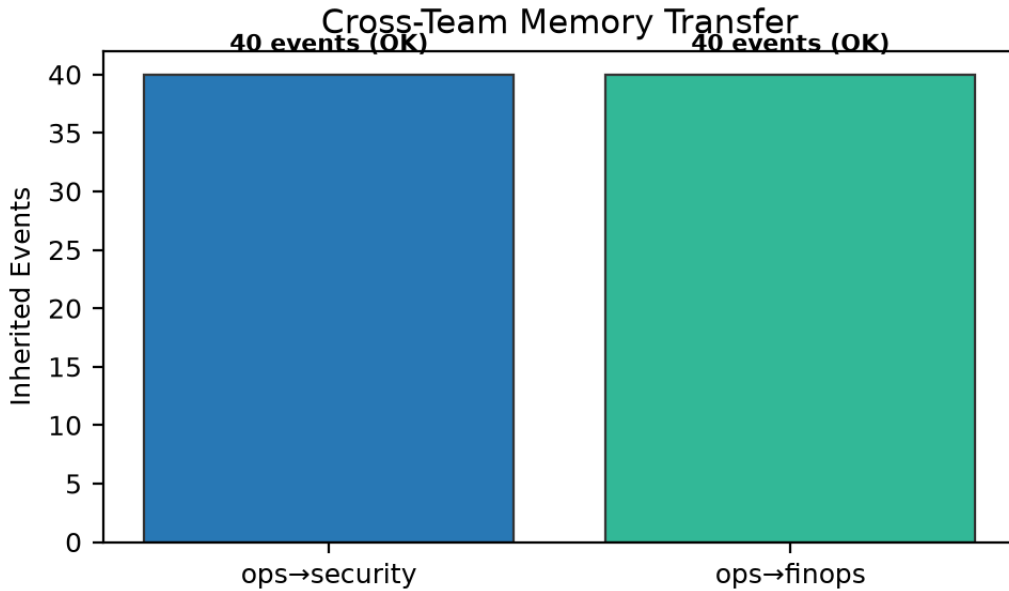


Figure 16: Cross-team memory transfer results.

7.13. Memory retention and controlled forgetting

This experiment tests whether memory consolidation and controlled forgetting can operate together during continued task execution. Forty episodic events with importance scores from 3 to 9 are injected into one agent and processed for five cycles (Table 12 and Fig. 17).

In every cycle, four high-importance episodes are abstracted into semantic claims. The semantic core grows from 4 items in cycle 1 to 8, 12, 16, and 20. New semantic abstractions are produced in all five cycles, and by cycle 5 the semantic core contains 20 claims, equivalent to 50% of the original episodic count.

Active episodes decrease only once, from 40 to 39 in the first cycle, and remain unchanged thereafter. Only one soft-forgetting event is triggered over the full experiment, a controlled-forgetting ratio of 2.5%. By cycle 5, 39 active episodes coexist with 20 semantic claims, showing that consolidation creates an abstraction layer without wholesale deletion of concrete experience.

Memory consolidation and controlled forgetting therefore operate jointly: high-importance episodes are progressively distilled into semantic knowledge, whereas a small number of low-value events are deactivated through soft forgetting. The resulting policy retains concrete experience, distills reusable semantics, and selectively compresses low-value memory.

Table 12: Five-cycle memory consolidation and controlled forgetting.

Cycle	Consolidated	Forgotten	Semantic core	Active episodes	Risk tendency
1	4	1	4	39	neutral
2	4	0	8	39	cautious
3	4	0	12	39	elevated
4	4	0	16	39	elevated
5	4	0	20	39	elevated

Memory Consolidation Dynamics (5-Cycle Trace)

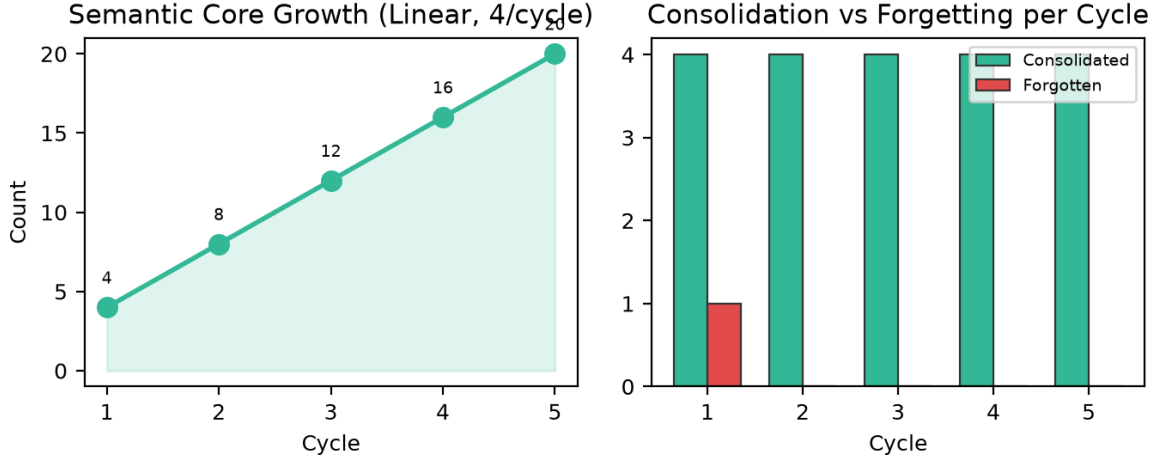


Figure 17: Memory consolidation dynamics and forgetting per cycle.

7.14. Robustness of the selection-inheritance chain

This experiment tests contamination screening, snapshot rollback, version-conflict handling, and malicious-candidate interception during cross-agent movement of memory and skills. Four scenarios are evaluated: contaminated-record injection, destructive operations, cross-version migration, and malicious candidate injection (Table 13 and Fig. 18). Contamination screening treats malicious records as the positive class and uses recall and precision, defined in Eqs. (28) and (29).

$$Recall_{poll} = \frac{TP}{TP + FN} \quad (28)$$

$$Precision_{poll} = \frac{TP}{TP + FP} \quad (29)$$

Ten malicious records are injected into 80 clean events. The system flags 20 records, including all 10 malicious records and 10 clean records, yielding 100% recall, 50% precision, and $F1 \approx 66.7\%$. No malicious record is missed before import, showing that the current screening rule prioritizes high-recall interception.

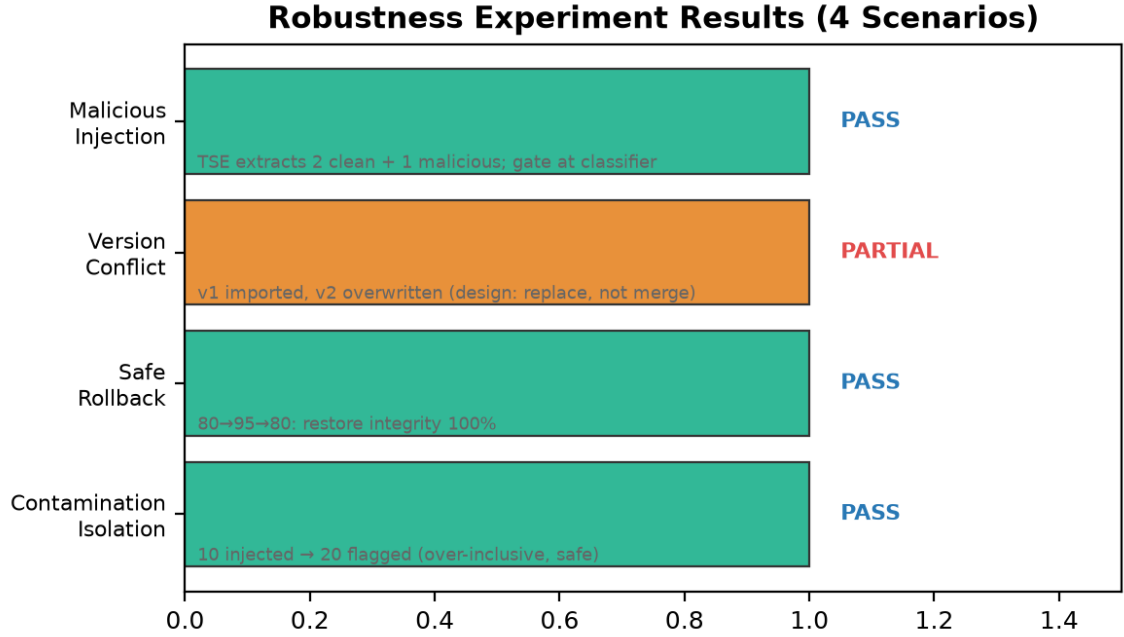
For snapshot rollback, 15 destructive operations are applied to an initial snapshot of 80 memory records. After `import_all`, the record count returns to the snapshot state, demonstrating a continuous recovery path from pre-transfer snapshot through transactional import to failure recovery. For cross-version migration, memory related to Terraform 0.12 is imported into an agent experienced with Terraform 1.5. The replace strategy performs only partial replacement within the compatible scope because some memory is incompatible with the target environment due to the version gap. Thus, version conflicts are processed deterministically according to policy while remaining constrained by target-environment compatibility.

The malicious-candidate injection scenario examines the separation between candidate generation and safety governance. TSE can transform malicious input into a schema-valid candidate object, allowing it to enter subsequent checks, but the candidate must still pass provenance, tool, execution, safety, and regression gates before formal publication. Schema validity alone therefore cannot bypass the release process.

All four protection paths operate in their intended roles. Contaminated records are identified before import, destructive operations can recover to the snapshot state, cross-version replacement is partially effective under compatibility constraints while conflicts remain policy-controlled, and malicious candidates remain subject to downstream validation gates. The system therefore exhibits preliminary layered robustness under the tested scenarios.

Table 13: Robustness scenarios and experimental results for the transfer and safety chain.

Scenario	Key result	Robustness conclusion
Contamination isolation	10 malicious records injected into 80 clean events; 10/10 malicious records flagged	Recall 100%; high-recall pre-import screening effective
Safe rollback	15 destructive operations applied to an 80-record snapshot; count restored after import_all	Snapshot, transaction, and recovery chain effective in this scenario
Cross-version replacement	Terraform 0.12 memory imported into a 1.5 environment; replace partially applied within compatible scope	Conflict policy executes deterministically; version gap limits replaceable scope
Malicious candidate injection	TSE generates a schema-valid candidate; safety gates block publication	Extraction and safety governance are separated; malicious candidates cannot be directly published

*Figure 18: Result matrix of four transfer- and safety-chain robustness tests.*

8. Conclusions and outlook

8.1. Conclusions

This paper starts from ecological selection as the basis of agent-skill evolution. Evolutionary character comes from differential persistence under task and environmental pressure, not from deliberation itself. Plaza and DART-Net/TSE act as a variation accelerator; the six gates and version competition perform human-constrained pre-selection; ecological pressure provides environmental selection; and the four-layer memory architecture with SWEI provides inheritance-like retention of selected capability, validity conditions, and failure boundaries.

Cluster-operations experiments show that the complete Plaza configuration improves candidate formation, DART-Net/TSE supports millisecond-scale extraction, lifecycle management and routing provide controlled selection, and four closed-loop cycles support bounded skill updating. SWEI and the safety chain preserve post-selection capability and recover safely from failures. The skill count increases from 5 to 9 and the internal quality score Q rises from 0.42 to 0.78.

The ecological-pressure experiment shows a sample-level inverted-U pattern in collaboration frequency, and one nine-generation mixed run produces a non-empty persistent skill combination. These findings show that environmental selection has been operationalized and yields recordable differential-persistence signals; ecology is

therefore a necessary selection element of the loop rather than an added observation. Nevertheless, four resource levels, mainly one- to two-generation runs, limited independent replication, and confounded variables leave a substantial evidence gap to broad claims about natural selection, causality, optimality, or convergence.

8.2. Outlook

Future work will broaden task types and agent populations, add independent random seeds and runs beyond ten generations, and separate resource abundance, generational depth, recombination, and population size. Survival analysis, event-history models, and intervention-based ablations should replace the current frequency analysis. Controlled inherited-versus-cold-start comparisons should then test whether skill combinations selected under ecological pressure improve later task performance. General claims about ecological selection, convergence, or optimality require repeated long-horizon and causally controlled experiments.

References

- [1] Du Y, Li S, Torralba A, Tenenbaum J B, Mordatch I. Improving factuality and reasoning in language models through multiagent debate [EB/OL]. arXiv:2305.14325, 2023.
- [2] Liang T, He Z, Jiao W, et al. Encouraging divergent thinking in large language models through multi-agent debate [C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Miami: Association for Computational Linguistics, 2024: 17889-17904.
- [3] Chen J, Saha S, Bansal M. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs [C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Bangkok: Association for Computational Linguistics, 2024: 7066-7085.
- [4] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation [EB/OL]. arXiv:2308.08155, 2023.
- [5] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta programming for a multi-agent collaborative framework [C]//International Conference on Learning Representations. 2024.
- [6] Park J S, O'Brien J, Cai C J, et al. Generative agents: Interactive simulacra of human behavior [C]//Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. New York: ACM, 2023: 1-22.
- [7] Robert H M III, Honemann D H, Balch T J, et al. Robert's Rules of Order Newly Revised, 12th ed. [M]. New York: PublicAffairs, 2020.
- [8] Landemore H. Open Democracy: Reinventing Popular Rule for the Twenty-First Century [M]. Princeton: Princeton University Press, 2020.
- [9] Kaner S, Lind L, Toldi C, Fisk S, Berger D. Facilitator's Guide to Participatory Decision-Making, 3rd ed. [M]. San Francisco: Jossey-Bass, 2014.
- [10] Bai S, Kolter J Z, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling [EB/OL]. arXiv:1803.01271, 2018.
- [11] Beltagy I, Peters M E, Cohan A. Longformer: The long-document transformer [EB/OL]. arXiv:2004.05150, 2020.
- [12] Sainz O, Garcia-Ferrero I, Agerri R, et al. GoLLIE: Annotation guidelines improve zero-shot information extraction [C]//International Conference on Learning Representations. 2024.
- [13] Lewis M, Liu Y, Goyal N, et al. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension [C]//Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020: 7871-7880.
- [14] Wang G, Xie Y, Jiang Y, et al. Voyager: An open-ended embodied agent with large language models [EB/OL]. arXiv:2305.16291, 2023.
- [15] Sumers T R, Yao S, Narasimhan K, Griffiths T L. Cognitive architectures for language agents [EB/OL]. arXiv:2309.02427, 2023.
- [16] Boyd R, Richerson P J. Culture and the Evolutionary Process [M]. Chicago: University of Chicago Press, 1985.
- [17] Brooks R A. Intelligence without representation [J]. Artificial Intelligence, 1991, 47(1-3): 139-159.
- [18] Gause G F. The Struggle for Existence [M]. Baltimore: Williams & Wilkins, 1934.
- [19] Lenski R E, Ofria C, Pennock R T, Adami C. The evolutionary origin of complex features [J]. Nature, 2003, 423: 139-144.
- [20] Ray T S. An approach to the synthesis of life [C]//Langton C G, Taylor C, Farmer J D, Rasmussen S, eds. Artificial Life II. Redwood City: Addison-Wesley, 1991: 371-408.
- [21] Visscher P M, Hill W G, Wray N R. Heritability in the genomics era—concepts and misconceptions [J]. Nature Reviews Genetics, 2008, 9: 255-266.
- [22] Stanfield R B. The Art of Focused Conversation: 100 Ways to Access Group Wisdom in the Workplace [M]. Gabriola Island: New Society Publishers, 2000.
- [23] Ching F D K, Jarzombek M M, Prakash V. A Global History of Architecture, 3rd ed. [M]. Hoboken: Wiley, 2017.
- [24] Robertson S, Zaragoza H. The probabilistic relevance framework: BM25 and beyond [J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.